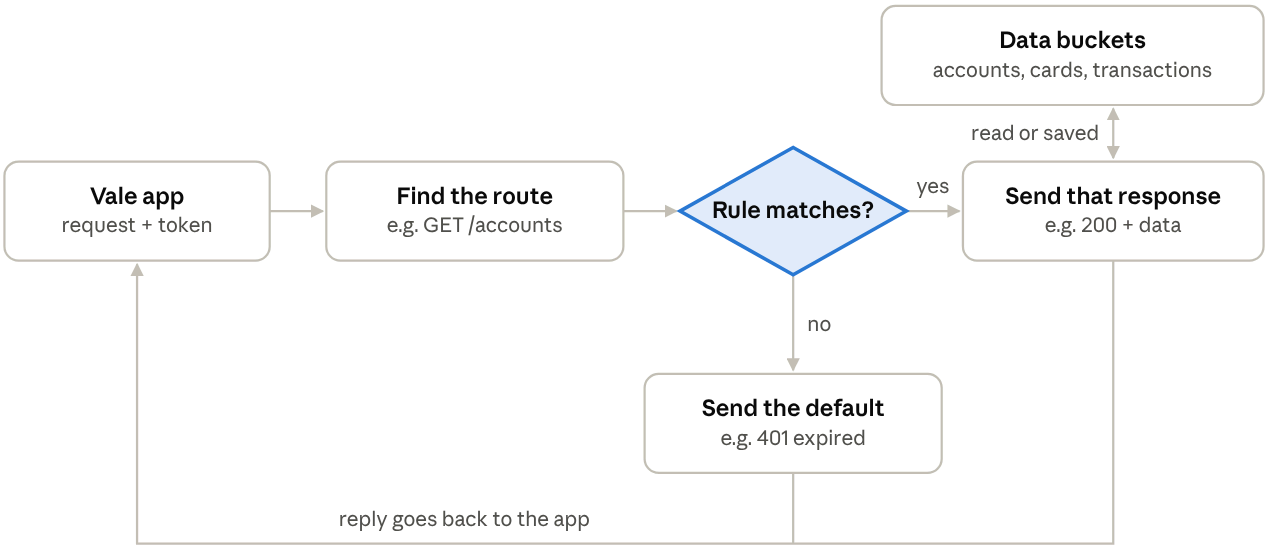
Vale API Overview

The Vale API is a pretend bank server that runs in Mockoon on `http://localhost:3000`. It answers the Vale Banking app's requests with ready-made data, so the app can be built and tested before a real backend exists.

How it works

Every request goes through the same three steps: Mockoon finds the **route** (method + path), checks that route's **rules**, then sends back a **response**.

Each request: find the route, check its rules, reply; buckets keep changes



How Mockoon answers a request

Each route has two or three possible responses. Mockoon checks their rules from top to bottom and sends the first one that matches, such as "the password is `Vale@2026`" or "the token is correct". If none match, it sends the **default**, which is usually an error. Some routes also read or save data in **data buckets**, so changes stay saved between requests.

Signing in

Signing in takes two requests, and the second one gives the app a token: a key it must show on every later request.

Step	Request	Send	Right answer	Wrong answer
1. Password	`POST /auth/login`	`{ "email", "password" }`	**200** with a `challengeId` and the phone's last 4 digits	**401** "Incorrect email or password."
2. Code	`POST /auth/verify`	`{ "challengeId", "code" }`	**200** with an `accessToken`	**401** "That code is incorrect."

The app saves the `accessToken` in the iOS Keychain. From then on it adds this header to every request:

```
Authorization: Bearer vale-demo-7f3a9c2e
```

Banking routes

The other eight routes are protected: they only answer if the request carries the right token. Without it, every one of them returns **401** "Your session has expired. Please sign in again.", which makes the app log the user out.

Request	What it does	Sends back	Used on
`GET /me`	Who is signed in	Name, email, last 4 digits of the phone	Home greeting, Profile
`GET /accounts`	My accounts	Everyday checking $8,450.80 and Savings $12,300.00	Home, Dashboard, Profile
`GET /cards`	My card	Debit card •• 4829, `active` or `locked`	Cards
`PATCH /cards/:id`	Lock or unlock the card	The card with its new status	Cards
`GET /transactions`	Payment history	Payments, newest first	Home
`GET /insights/spending`	This month's spending	Total, weekly amounts, categories, net cash flow	Dashboard

Request	What it does	Sends back	Used on
`POST /transfers`	Move money between my accounts	**201** with the completed transfer	Transfer sheet
`POST /auth/logout`	End the session	**204**, no body	Profile

What gets saved

Three **data buckets** remember changes while Mockoon is running. A data bucket is like a small table: some routes read from it, others write to it, so a change made by one request shows up in the next.

Bucket	Holds	Read by	Changed by
`accounts`	Both balances	`GET /accounts`	`POST /transfers` takes the amount from one balance and adds it to the other, rounded to cents
`cards`	Card status	`GET /cards`	`PATCH /cards/:id` saves `locked` or `active`
`transactions`	Payment history	`GET /transactions`	`POST /transfers` adds a "To Savings" or "From Savings" payment

For example, after a $100.10 transfer from checking, `GET /accounts` returns $8,350.70 for checking and $12,400.10 for savings, and the transfer is at the top of `GET /transactions`.

Press ■ then ▶ in Mockoon to reset every bucket to its starting data. You can see the starting values in Mockoon's **Data** tab.

Demo values and error tests

Sign in with any email, password `Vale@2026` and code `123456`.

To test	Do this	The API answers
Wrong password	Any other password	401 Incorrect email or password

To test	Do this	The API answers
Wrong code	Any code except `123456`	401 That code is incorrect
Declined transfer	Transfer note `decline`	422 This transfer was declined; nothing is saved
Expired session	In Mockoon, change the token in a route's rule, then refresh the app	401, and the app logs out
Server down	Press ■ in Mockoon, then refresh the app	No answer; the app shows "Can't reach Vale right now"
Quick check	Open http://localhost:3000/me in a browser	401, because the browser has no token

Mockoon's **Logs** tab shows every request the app sends and the response Mockoon picked, which is the fastest way to see what went wrong.

Limits compared with a real bank server

The Vale API	A real bank server
Saves changes only while Mockoon runs; restart resets them	Saves to a database permanently
One fixed token that never expires	A new short-lived token per sign in, refreshed and revocable
Accepts any email and doesn't check the amount against the balance	Checks every input and every limit on the server
Plain `http` on your Mac	HTTPS only, often with certificate pinning
`/me` and spending numbers never change	Calculated from real activity

These limits are fine for building and testing the app, because the app talks to the mock exactly as it would talk to a real server. Moving to a real backend would mean changing the base URL, not the app's code.